

CHIA-Governor: Cost Governance for Agentic Hardware Design Loops, and a Small Model That Knows When Not to Simulate

Venkatesh Rajagopal

Independent · venkatesh.rajagopal@gmail.com

ABSTRACT

Agentic hardware design loops spend a large share of their budget on iterations that produce nothing. We present **CHIA-Governor**, three reusable blocks for the CHIA framework that reduce that waste without changing the result a loop produces: a cost ledger, a triage gate that blocks doomed candidates before simulation, and a plateau monitor that stops a loop once improvement per dollar flatlines. To populate the gate we build a dataset of 2,571 Verilog candidates whose pass/fail labels come from actually running VerilogEval test benches, and fine-tune Qwen2.5-Coder-7B on it with LoRA in 72 minutes on a single NVIDIA L4. The resulting model reaches **83.3% precision** on predicting simulation failure, higher than Gemini 2.5 Flash (77.9%), Pro (71.5%) and Flash-Lite (59.3%), at roughly a quarter of Flash's cost and a twentieth of its latency. The gain comes from a specific, diagnosable place: every frontier model we tested assumes that any edit to working code is a bug, and therefore discards between 23% and 56% of genuinely good designs. Our model discards 14%.

1. THE PROBLEM

A CHIA loop repeats a cycle: an agent proposes a change to a design, tools evaluate it, and the loop decides whether to continue. The published CHIA results show how much of that cycle is wasted. The critical-path optimization study ran 14 iterations at roughly \$14.43 each but had already plateaued after the eighth, so approximately half of its \$202.03 bought nothing. The gem5-alignment study ran 202 iterations for \$2,355.98.

Two distinct wastes are visible here. The loop does not know when to stop, and it sends every proposed candidate to the expensive evaluation stage regardless of how likely that candidate is to fail. CHIA is an excellent framework for *expressing* such loops. It provides nothing reusable for *governing* what they cost.

The framework's own most striking positive finding points the same direction: adding a single "Run & Compare" tool, letting the agent empirically test its own changes, doubled the improvement rate per iteration. Feedback quality and spend discipline, not iteration count, determine whether a loop converges.

2. CHIA-GOVERNOR

We contribute three composable blocks. Each is useful alone; together they address both wastes above.

CostLedger records every charge against the node and iteration that incurred it, billing either token usage or wall-clock time on an hourly machine, with optional hard budget enforcement. It is the measurement substrate the other two blocks require and is independently useful to anyone working against a fixed budget.

TriageGate sits in front of the expensive evaluation stage and asks a cheap model whether a candidate will pass its tests. Crucially it returns a probability rather than a verdict, because the correct operating point depends on the relative cost of a wasted simulation versus a discarded working design. Scorers are pluggable: a local fine-tuned model, a frontier API, or a heuristic.

PlateauMonitor tracks improvement in the objective per dollar spent and halts the loop when that stays below a caller-set threshold for several consecutive iterations. Patience matters: one flat round is noise, three is a plateau.

2.1 What the monitor would have saved

Replaying CHIA's own published critical-path trajectory through PlateauMonitor requires no compute and gives a concrete number.

Table 1: Counterfactual on the published critical-path run (14 iterations, \$202.03, 47 – 95 MHz).

Patience	Stops at	Spend	Saved	Result
1	iter 9	\$129.87	35.7%	95 MHz
2	iter 10	\$144.30	28.6%	95 MHz
3	iter 11	\$158.73	21.4%	95 MHz

The final result is unchanged in every case, because the skipped iterations are precisely the ones that produced nothing.

3. BUILDING THE GATE'S TRAINING DATA

The hard part of training a triage model is obtaining labels. We avoid collecting them by generating data whose labels are correct by construction, then confirming every one against a real simulator.

We start from the 156 problems of VerilogEval, each of which supplies a natural-language specification, a correct reference implementation, and a test bench that reports mismatches against that reference. We apply scripted edits to each reference, then compile and simulate every variant with Icarus Verilog. The label is whatever the test bench actually reports, never an assumption.

3.1 Two families of edit, and why both are needed

Our first attempt used only edits that break behaviour: flipped comparisons, swapped operators, off-by-one constants, altered bit ranges, changed reset values, non-blocking converted to blocking, dropped assignments, swapped signals. The resulting dataset was **79.7% FAIL**. A model scores 80% on such a set by answering FAIL every time, which makes the benchmark meaningless.

We therefore added six *behaviour-preserving* operators that mimic edits an agent plausibly makes without breaking anything: commutative operand swaps, consistent internal signal renames, redundant parentheses, reformatted literals (4'd10 to 4'b1010), reordered independent continuous assignments, and unused wire declarations. We also capped how often any single operator may fire per problem, after observing that one easy-to-spot edit had produced two thirds of the harmless examples and could be memorised.

This rebalancing is not cosmetic. It changes the question the model is asked from "is this code broken" to "is this change harmful or harmless", which is the question an agentic loop actually faces.

Table 2: Dataset composition after rebalancing.

Property	Value
Source problems	156
Candidates generated	3,171
Compiled successfully (usable)	2,571
FAIL share	44.8%
Train / test split	1,941 / 630
Problem overlap across split	0

Candidates that fail to compile are discarded: the compiler already catches those for free, so they are not what a gate exists to predict. The split is **by problem, not by example**, so the model is always evaluated on designs it has never seen in any form. Within the test set, 289 candidates fail and 341 pass, making the majority-class baseline 54.1%.

4. EXPERIMENTAL SETUP

We fine-tune Qwen2.5-Coder-7B-Instruct with LoRA (rank 16, 40.4M trainable parameters, 0.53% of the model) loaded in 4-bit, for two epochs. Training took 72 minutes on one NVIDIA L4. Loss fell from 0.23 and plateaued near 0.10 rather than collapsing toward zero, which is the shape of a learned decision rule rather than memorisation.

Every model, local and frontier, receives the identical prompt: the natural-language specification plus the candidate implementation, and is asked for one word. Local models are scored by comparing the logits of the PASS and FAIL tokens at the answer position, which is deterministic, cannot produce an unparseable answer, and yields a probability the gate can threshold.

We report **precision on FAIL** as the primary measure. A false FAIL silently discards a working design, which is worse for a design loop than paying for one additional simulation. Recall indicates how many simulations are saved.

5. RESULTS

Table 3: All models on the same 630 held-out examples. Positive class is FAIL.

Model	Acc	Prec	Rec	\$/1k	Lat
Qwen 7B, untrained	61.4%	56.1%	73.4%	0.049	0.25s
Qwen 7B, tuned	85.2%	83.3%	84.8%	0.063	0.32s
Gemini 2.5 Flash	86.2%	77.9%	97.6%	0.163	6.47s
Gemini 2.5 Pro	81.5%	71.5%	99.7%	0.676	14.4s
Gemini 2.5 Flash-Lite	68.1%	59.3%	96.9%	0.054	0.42s

Fine-tuning moves accuracy from 61.4% to 85.2% and precision from 56.1% to 83.3%. Since the base model, the tuned model, the prompt and the test set are otherwise identical, that difference is attributable to the training data.

Against the frontier, the tuned 7B has **the highest precision of any model tested**, at 2.6× lower cost and 20× lower latency than Gemini 2.5 Flash. Flash retains clearly better recall and marginally better raw accuracy. We also note that Flash outperforms Pro on this task despite costing a quarter as much, suggesting that additional reasoning does not help and may actively hurt.

5.1 Why: every frontier model over-blocks

Recall on FAIL between 96.9% and 99.7%, paired with precision between 59.3% and 77.9%, is the signature of a model that

condemns nearly everything. Splitting accuracy by edit family shows where that comes from.

Table 4: Accuracy by edit family. Benign edits preserve behaviour.

Model	Benign	Faulty
Qwen 7B, untrained	55.6%	65.6%
Qwen 7B, tuned	94.8%	79.4%
Gemini 2.5 Flash	83.7%	88.5%
Gemini 2.5 Pro	74.5%	86.9%
Gemini 2.5 Flash-Lite	49.2%	83.5%

The untrained model sits near chance on harmless edits. Flash-Lite, at 49.2%, is *below* chance: it has effectively learned to answer FAIL unconditionally. Every model does markedly better on faulty edits than benign ones. The prior is consistent and wrong: a change to working code is assumed to be a bug.

Fine-tuning on data that contains *verified-harmless* edits alongside real faults moves benign accuracy from 55.6% to 94.8%, a 39-point gain, while faulty accuracy also improves. This is the mechanism behind the headline number, and it was predicted before the experiment rather than discovered afterwards.

5.2 What this means for a loop

Table 5: Gate economics on 630 candidates (341 genuinely good).

Model	Sims skipped	Good lost
Qwen 7B, untrained	378	48.7%
Qwen 7B, tuned	293	14.1%
Gemini 2.5 Flash	362	23.5%
Gemini 2.5 Pro	403	34.1%
Gemini 2.5 Flash-Lite	472	56.3%

Because the gate emits a probability, its operating point is tunable. At a threshold of 0.90 the tuned model reaches 92.7% precision while discarding only **14 of 341** good designs (4.1%) and still skipping 192 simulations. At 0.95 it reaches 95.0% precision and loses 8. A deployer who values working designs highly can move up this curve; one who values throughput can move down it.

Table 6: Threshold sweep, tuned model.

Thresh	Acc	Prec	Rec	Blocked	Good lost
0.30	82.5%	75.4%	92.0%	353	87
0.50	85.2%	83.3%	84.8%	294	49
0.70	83.7%	86.9%	75.8%	252	33
0.90	80.2%	92.7%	61.6%	192	14
0.95	77.0%	95.0%	52.6%	160	8

6. LIMITATIONS

We report these plainly because a gate that is trusted incorrectly is worse than no gate.

Recall is our weakest dimension. At 84.8% we miss roughly one failing candidate in six, which Flash catches. A deployment that cannot tolerate wasted simulations should prefer Flash, or chain the two.

Faults are synthetic. Our mutations imitate plausible agent errors but are not drawn from real agent output. A loop's actual mistakes may be distributed differently, and performance on them is unmeasured.

One benchmark, small modules. VerilogEval modules are far smaller than a BOOM pipeline stage. Whether the result transfers to industrial-scale RTL is untested.

Contamination is plausible. VerilogEval predates these models and may appear in their training data. Our mutated variants do not, but the underlying problems may, which would favour the frontier baselines.

Single run, no seeds. Time constraints permitted one training run. We report no variance across seeds.

Benign is not perfectly benign. Five of 252 supposedly harmless edits did fail simulation, where a reordering was not in fact independent. Labels come from the simulator, so these are correct, but the family name is approximate.

Cost figures are indicative. Local cost assumes \$0.70/hour for an L4 at measured throughput; frontier cost uses published per-token pricing. Both move.

7. AVAILABILITY

Everything is released under BSD-3-Clause, matching CHIA.

Code, dataset and results:

github.com/venkateshrajag/chia

LoRA adapter:

[huggingface.co/emailvenky/
chia-governor-triage-7b](https://huggingface.co/emailvenky/chia-governor-triage-7b)

The repository contains the three blocks, the full generation and evaluation pipeline, all 2,571 labelled candidates with the train and test splits, and the raw predictions from every model reported here. It also retains the discarded 79.7%-FAIL first dataset, so the rebalancing argument in Section 3.1 can be checked rather than taken on trust. Every label is reproducible by re-running the included simulation harness against VerilogEval, and one script regenerates every table in this paper from the raw prediction files.

8. CONCLUSION

Agentic design loops waste money in two ways that are both fixable with small, reusable components: they run past the point of improvement, and they pay to evaluate candidates that were never going to work. We provide blocks for both, and show that the model inside the gate need not be large or expensive. A 7B model, fine-tuned for 72 minutes on one commodity GPU using data generated and labelled in under two minutes, predicts simulation failure more precisely than any frontier model we tested, because it was trained against the specific bias those models share. The total compute cost of every experiment in this paper was under ten dollars.